ENGINEERING PORTFOLIO

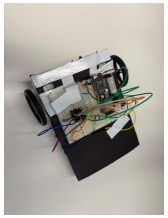
Ariadne Colina-Valeri

Hi! My name is Ariadne, and I'm a Junior studying Electrical Engineering at Tufts University. I have experience in signal processing, embedded systems and machine learning, from restoring RF signals with a Wiener filter to building a real-time audio effects board and an autonomous robot. I'm looking for internships in signal processing, embedded systems and machine learning.

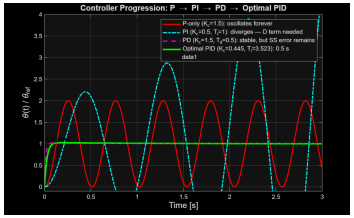


ariadnecolina@gmail.com
(813) 847-5642

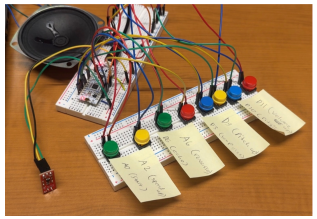
SELECTED PROJECTS



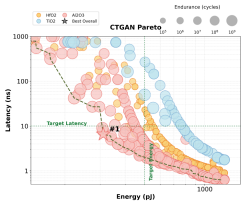
Autonomous Robot
page 2



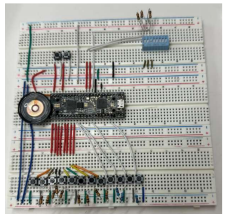
UAV Pitch Control
page 3



DJ Board on an STM32
page 4



CTGAN for RRAM
page 5



FPGA Synthesizer
page 6

PROJECT 1 | IN PROGRESS | JUNIOR DESIGN | SEPT. 2026 TO PRESENT

Autonomous ESP32 Robot

THE CHALLENGE

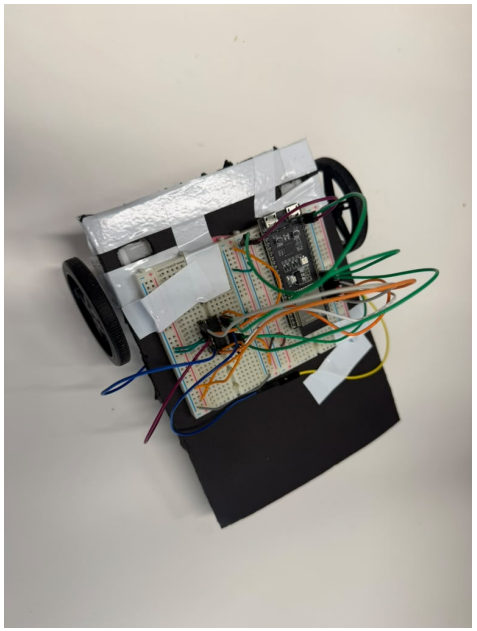
The robot has to follow a colored lane and avoid obstacles with no one controlling it, and accept commands only from our own team.

WHAT I'M DOING

I'm building a 7-state state machine on the ESP32 that authenticates every websocket command with our team key and rejects the rest. For obstacles, an IR LED and photodetector measure reflected light, and I subtract the room's background light from each reading. For the lane, I filter noise from the color sensor before the robot decides which color it is on.

NEXT

A ladybug-shaped body I'm designing in SolidWorks, and a custom ESP32 PCB with parts chosen from datasheets to replace the breadboard wiring.



The robot so far, wired on a breadboard around the ESP32



The ladybug body I'm designing in SolidWorks

PROJECT 2 | LINEAR SYSTEMS | JAN. 2026 TO MAY 2026

UAV Pitch Control and RF Signal Restoration

THE CHALLENGE

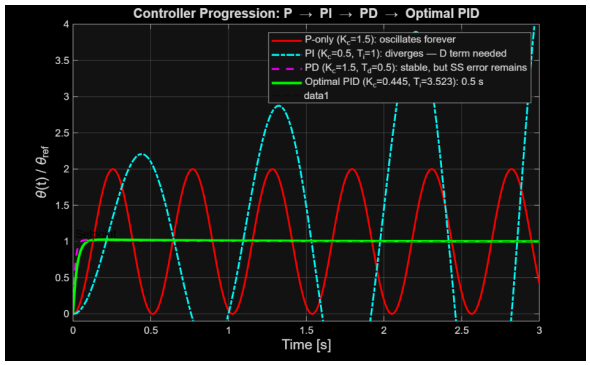
A UAV that shakes while collecting RF signals blurs them, so it has to stay level before the signals can be trusted.

WHAT I DID

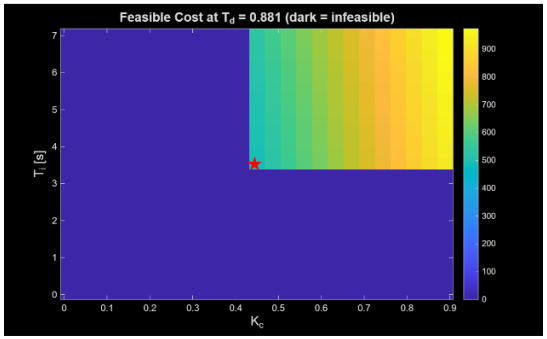
I proved with Routh-Hurwitz analysis that P and PI controllers can never stabilize it, then searched 15,625 PID gain combinations and refined the grid 10 times. I restored a blurred RF signal (radar, chirp and communications components at 40dB SNR) with a generalized Wiener filter in MATLAB, and linked the two parts: the controller's settling time sets how long the blur lasts.

THE RESULT

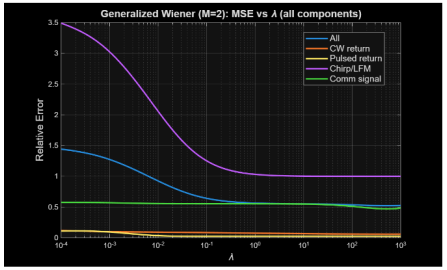
A 52ms rise time, 2.6% overshoot and 88.3° phase margin, 77% less error than a direct inverse, and proof that a controller settling in 2s instead of 0.5s makes recovery 1.5× worse.

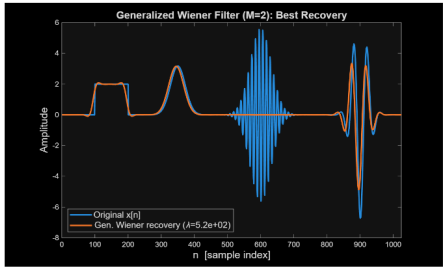


P, PI, PD and PID controllers



Cost of each gain pair; the star is the optimum





Wiener filter error (left) and the restored RF signal over the original (right)

DJ Board on an STM32

(Audio Recorder and Effects Processor)

THE CHALLENGE

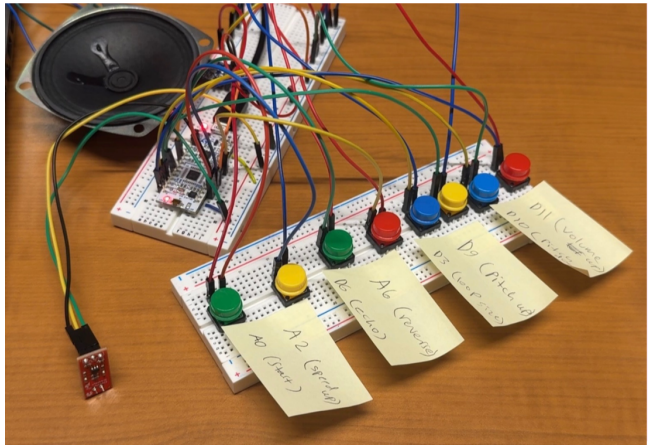
Seven real-time audio effects had to fit in the STM32's 64KB of on-chip memory, and the first playback came out with a high-pitched hiss and clicks.

WHAT I DID

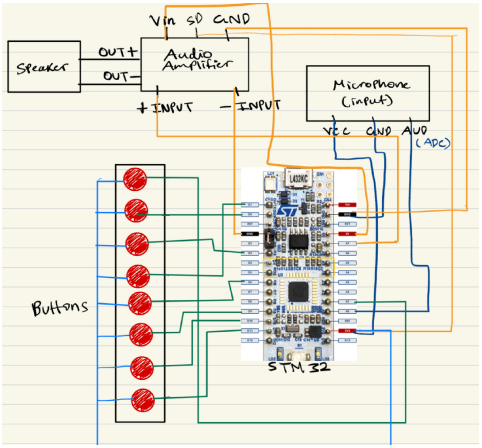
To remove the hiss, I used a 4kHz timer interrupt to record and play every sample at the same rate, and lengthened the ADC sampling time to 12.5 cycles so the microphone signal settles before each reading. The clicks came from sudden jumps in the speaker signal, so I held the output at mid-rail between clips and faded each clip in and out over 256 samples (64ms). Dropping from 8kHz to 4kHz stretched each recording from 2 to 5 seconds.

THE RESULT

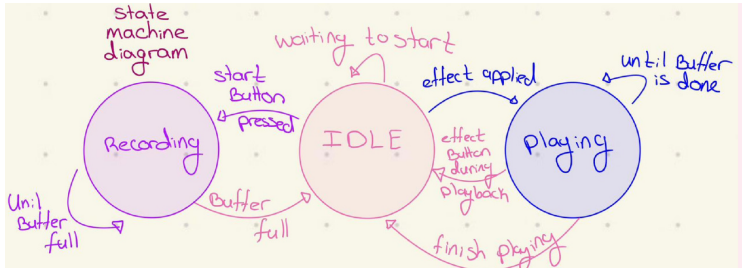
5 seconds of 12-bit audio in a 40KB buffer, all 7 effects (echo, reverse, speed, loop size, pitch up, pitch down, volume), and a playback waveform that matched the original on the oscilloscope.



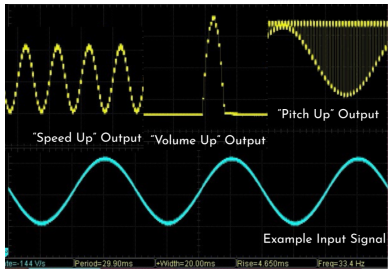
The board



Hardware: wiring diagram



Software: the state machine



Oscilloscope captures of the effects

STM32: every sample moves on a 4kHz timer



CTGAN Design Discovery for RRAM

THE CHALLENGE

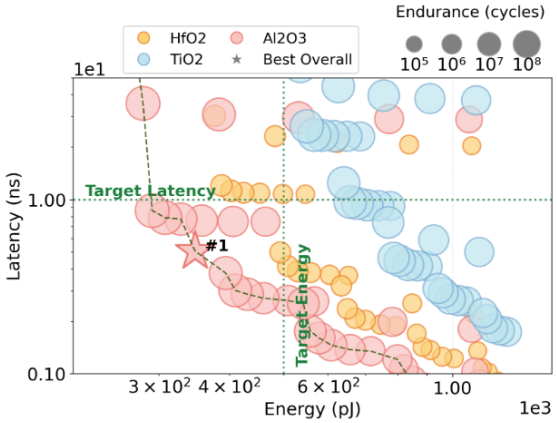
The published framework I extended used a CVAE, which averages its training data and smooths away the extreme designs where the best performance is found.

WHAT I DID

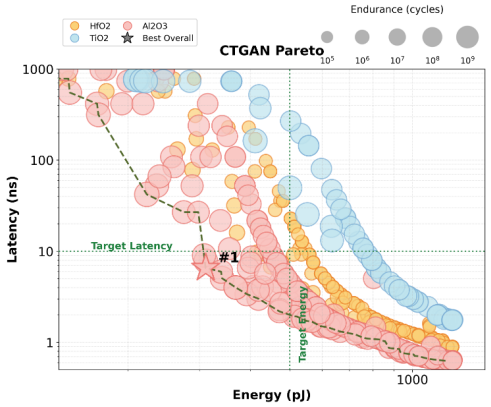
I replaced it with a CTGAN that searches those extremes more aggressively and used the framework's physics model (PINN) as a referee to reject impossible designs. The CTGAN collapsed at 500 training samples and stayed noisy at 2,500, so I grew the data to 5,000, where it converged across 3 oxides (Al_2O_3, HfO_2 and TiO_2).

THE RESULT

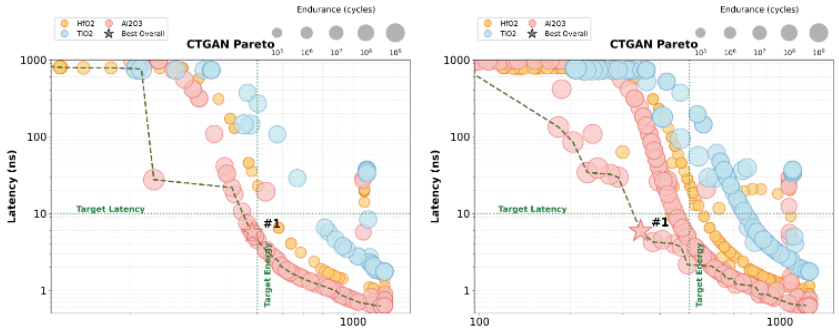
An Al_2O_3 design with 4.84×10^9 cycles against the published 3.85×10^9 (25% higher endurance) and 310.7pJ against 350.6pJ (11% less energy), with slower switching (6.57ns vs 4.07ns).



Published CVAE results



My CTGAN with 5,000 samples



The CTGAN with 500 samples (left) and 2,500 (right)

PROJECT 5 | DIGITAL LOGIC CIRCUITS | SEPT. 2025 TO DEC. 2025

FPGA Audio Synthesizer

THE CHALLENGE

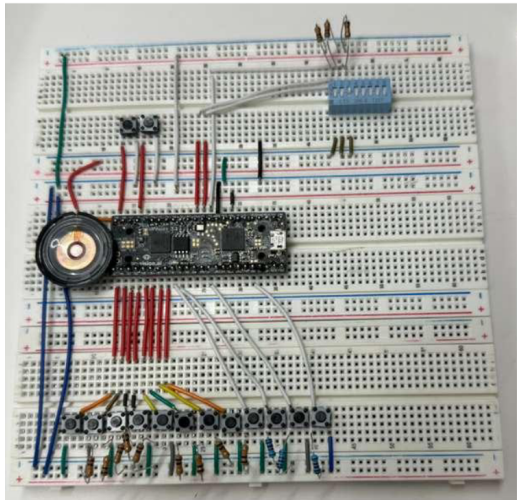
Each octave doubles a note's frequency, and my first design calculated every note by dividing on the chip, which needed over 250% of the FPGA's space. Chords of more than 2 notes, or in low octaves, turned into static.

WHAT I DID

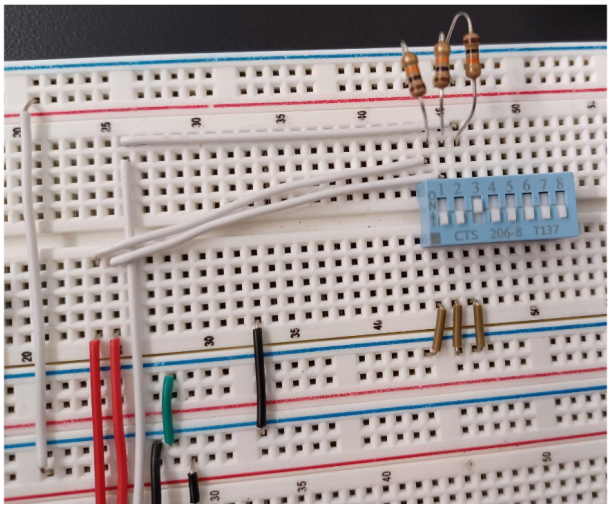
A tuner showed every note playing an octave low, which meant the clock ran at twice the rate in the code. Since dividing by 2 is the same as shifting bits, I hand-calculated one octave, used bit shifts for the rest, and widened the period counter so notes below 92Hz could play. To find the static, I plotted C and E summed in octave 3 and octave 7 in Desmos, then mixed the notes with PWM on a 3-bit bus, the best balance of volume and clarity.

THE RESULT

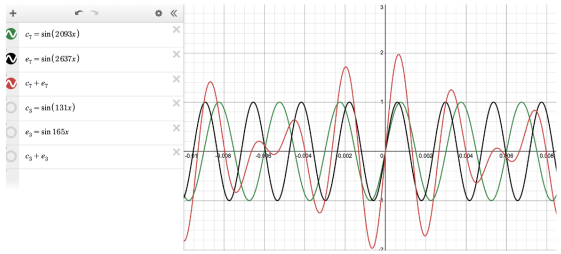
All 12 notes across 8 octaves (16Hz to 8kHz), chords of 5 to 6 notes in the lowest octave, and every note at once in the highest.



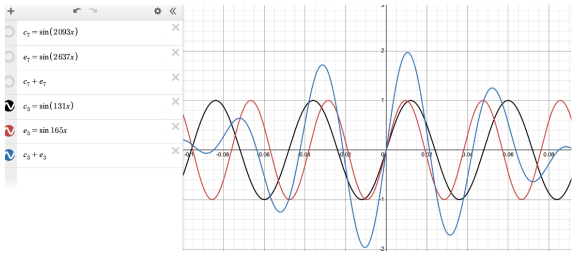
The complete board



The switch that selects the octave



C + E summed in octave 7 (red)



C + E summed in octave 3 (blue)